X-CAVATE Refactoring Report

Parity Fixes: Achieving Functional Equivalence Between
`xcavate_11_30_25.py` and the Refactored `xcavate/` Package

March 8, 2026

Test Network: `tree_Test2.txt` (1,100 points across 11 vessels; 31,271 after interpolation)

Contents

1 Executive Summary **3**
- 1.1 Key Result . . . 3

2 Architecture Overview **3**
- 2.1 Original: Monolithic Script . . . 3
- 2.2 Refactored: Modular Package . . . 3

3 Fixes Implemented **4**
- 3.1 Fix 1: Artven Noise Swap Bug (**HIGH**) — `pipeline.py` . . . 4
- 3.2 Fix 2: SM G-code Hardcoded `Z` Axis (**HIGH**) — `pressure.py` . . . 4
- 3.3 Fix 3: SM Pressure Pass Transition G91/G90 Wrapping (**HIGH**) — `pressure.py` . . . 5
- 3.4 Fix 4: Overlap Algorithm (**HIGH**) — multiple files . . . 5
- 3.5 Fix 5: Multimaterial Classification Node (**MEDIUM**) — `multimaterial.py` . . . 6
- 3.6 Fix 6: Print Instructions Block (**HIGH**) — `pipeline.py` . . . 6
- 3.7 Fix 7: Original Network Plot (**LOW**) — `pipeline.py` . . . 7
- 3.8 Fix 8: Flow Rate Default (**MEDIUM**) — `config.py`, `cli.py`, `gui/app.py` . . . 7
- 3.9 Fix 9: MM Positive Ink Overshoot Adjustment (**MEDIUM**) — `base.py` . . . 7
- 3.10 Fix 10: Aerotech SM Mode (**LOW**) — `aerotech.py` . . . 8
- 3.11 Fix 11: Downsampled and Overlap Plots (**LOW**) — `pipeline.py` . . . 8

4 Intentional Differences Retained **8**
- 4.1 4A. Iterative DFS (Issue 2G) . . . 8
- 4.2 4B. Nearest-Neighbor Pass Reordering . . . 9
- 4.3 4C. KD-tree Inlet/Outlet Matching (Issue 2H) . . . 9

5 Files Modified **9**

6 Validation Results **9**
- 6.1 Test Configuration . . . 9
- 6.2 Coordinate Coverage . . . 10
- 6.3 G-code Structure . . . 10
- 6.4 Speed Values . . . 10
- 6.5 Output Files . . . 10

6.6 Performance 10

7 Known Remaining Differences **10**

8 Improvements in the Refactored Package **11**

1 Executive Summary

The original X-CAVATE codebase was a single 5,568-line Python script (`xcavate_11_30_25.py`). It was refactored into a modular package (`xcavate/`) spanning 6,201 lines across 27 modules. A systematic comparison identified **14 functional gaps and behavioral differences** between the two codebases, ranked by severity. This report documents the **11 fixes** implemented to achieve functional parity, the **3 intentional differences** retained as improvements, and the **validation results** confirming equivalence.

1.1 Key Result

After all fixes, running both versions on the same test network produces:

Metric	Original	Refactored	Match?
Unique (X,Y,Z) coordinates	31,271	31,271	Exact
G-code line count	31,822	31,822	Exact
Unique speed values	436	436	Exact
Speed range (mm/s)	0.25 – 15.676	0.25 – 15.676	Exact
Print instructions	X=181.45, Y=152.9, Z=−271.82	X=181.45, Y=152.9, Z=−271.82	Exact
G-code file size	1,556,908 bytes	1,556,625 bytes	∼0.02%
Execution time	∼5 min (with recursion limit increase)	∼2 sec	150× faster

Table 1: Validation comparison between original and refactored code.

2 Architecture Overview

2.1 Original: Monolithic Script

`xcavate_11_30_25.py` (5,568 lines)

```
Lines 1-100:     Argument parsing
Lines 101-171:   Global variables (30+)
Lines 172-392:   File I/O and preprocessing
Lines 393-955:   Graph construction
Lines 956-980:   Recursive DFS pathfinding
Lines 981-3200:  Gap closure pipeline
Lines 3201-3806: Post-processing (downsample, overlap)
Lines 3807-5487: G-code generation (6 copy-pasted variants)
Lines 5488-5568: Print instructions
```

2.2 Refactored: Modular Package

`xcavate/` (6,201 lines across 27 files)

```
__init__.py              (3 lines)    Package init
__main__.py              (6 lines)    Entry point
cli.py                   (192 lines)  Backward-compatible CLI
config.py                (130 lines)  XcavateConfig dataclass
```

```
pipeline.py              (619 lines)  Pipeline orchestrator

core/                                 Core algorithms
    gap_closure.py       (950 lines)
    graph.py             (835 lines)
    pathfinding.py       (619 lines)  Iterative DFS + sweep-line
    postprocessing.py    (290 lines)  Subdivision, overlap, reorder
    preprocessing.py     (251 lines)
    multimaterial.py     (97 lines)

io/                                   Input/output
    reader.py            (298 lines)  KD-tree fuzzy matching
    writer.py            (147 lines)
    gcode/                            G-code generation
        base.py          (182 lines)  Abstract base class
        pressure.py      (152 lines)
        positive_ink.py  (183 lines)
        aerotech.py      (196 lines)

gui/app.py               (791 lines)  Streamlit web GUI
spatial/index.py         (119 lines)  KD-tree collision detection
viz/plotting.py          (136 lines)  Plotly 3D visualization
```

3 Fixes Implemented

3.1 Fix 1: Artven Noise Swap Bug (HIGH) — `pipeline.py`

Problem: In `_subdivide_by_material()`, the artven noise swapping modified a local Python list but never wrote the corrected values back to the `points` array. The break point detection on the next loop re-read from `points[n, 4]`, reading the *original unswapped values*. The entire noise removal was effectively a no-op.

Root cause: Lines 354–369 built a local `artven` list and swapped values in it, but lines 372–383 created a new `artven` list by re-reading `points[n, 4]`.

Fix: After the swapping loop, write corrected values back to the points array:

```
# After swapping loop (line 369), added:
for j, n in enumerate(nodes):
    points[n, 4] = artven[j]
```

Impact: Material boundary detection in multimaterial mode now correctly ignores noise at pass boundaries, matching the original script's behavior.

3.2 Fix 2: SM G-code Hardcoded `Z` Axis (HIGH) — `pressure.py`

Problem: Single-material pressure mode hardcoded `Z` as the axis name in all G-code commands (`G92 X{x} Y{y} Z{z}`). The original used `printhead1_axis` (configurable, defaults to `"A"` for Allevi printers).

Fix: Replaced all 6 hardcoded `Z` references with `cfg.axis_1`:

Line	Before	After
51	`G92 X{x} Y{y} Z{z}`	`G92 X{x} Y{y} {cfg.axis_1}{z}`
54	`G1 X{x} Y{y} Z{z} F{speed}`	`G1 X{x} Y{y} {cfg.axis_1}{z} F{speed}`
64	`G1 X{x} Y{y} Z{z}`	`G1 X{x} Y{y} {cfg.axis_1}{z}`
126	`G1 X{x} Y{y} Z{z} F{speed}`	`G1 X{x} Y{y} {cfg.axis_1}{z} F{speed}`
138	`G91 G1 Z{initial_lift}`	`G1 {cfg.axis_1}{initial_lift}`
139	`G90 G1 Z{network_top}`	`G1 {cfg.axis_1}{network_top}`

Table 2: Axis name replacements in `pressure.py`.

Impact: SM G-code now uses the configured axis name (e.g., `A` for Allevi), matching old behavior.

3.3 Fix 3: SM Pressure Pass Transition G91/G90 Wrapping (HIGH) — `pressure.py`

Problem: The refactored SM pass transition was missing the `G91/G90` (relative/absolute mode) wrapping around the custom extrusion start code.

Old code pattern (lines 4455–4476):

```
G90
; [stop extrusion]
G1 X Y                    ; jog without extrusion
G90
G1 X Y A(z)               ; lower to position
G91                        ; switch to relative mode
; [start extrusion custom code]
G90                        ; back to absolute
G1 X Y A(z) F(speed)      ; resume printing
```

Refactored (before fix):

```
G1 X Y                    ; jog
G1 X Y Z(z)               ; lower
; [start extrusion]        ; NOT wrapped in G91/G90
G1 X Y Z(z) F(speed)
```

Fix: Added `G90` → `G1 jog` → `G90` → `G1 lower` → `G91` → `[custom code]` → `G90` → `G1 print` pattern, and moved stop extrusion to `_write_pass_end()` with `G91` wrapping.

Also fixed: Pass end now uses hardcoded `F10` for jog-to-top (matching old `F10` vs the refactored `F{jog_speed}`).

3.4 Fix 4: Overlap Algorithm (HIGH) — multiple files

Files: `postprocessing.py`, `config.py`, `cli.py`, `gui/app.py`, `pipeline.py`

Problem: The overlap algorithms were fundamentally different:

Fix: Implemented both algorithms behind a selectable enum:

Aspect	Original (retrace)	Refactored (consecutive)
Scan scope	ALL previous passes	Only adjacent pairs
Match criterion	Last node appears anywhere in earlier pass	Last node is graph neighbor of next pass's first node
Attachment	Retrace backwards, append to END	Prepend to START of next pass

Table 3: Overlap algorithm differences.

```
class OverlapAlgorithm(Enum):
    RETRACE = "retrace"        # Original behavior (default)
    CONSECUTIVE = "consecutive" # Fast alternative
```

The `RETRACE` algorithm replicates the original exactly:

1. For each pass `i` (from 1 onward), get its last node
2. Search ALL passes `j < i` for that node
3. If found at index `idx` in pass `j`, retrace backwards: append nodes `[idx-1, idx-2, ...]` from pass `j` to the **end** of pass `i`
4. Up to `num_overlap` nodes appended

CLI: `-overlap_algorithm retrace|consecutive`
GUI: Selectbox in Advanced section (only visible when overlap > 0)
Default: `retrace` (matches original)

3.5 Fix 5: Multimaterial Classification Node (MEDIUM) — `multimaterial.py`

Problem: The refactored code classified each pass's material type using the **second node** (`nodes[1]`), while the original used the **last node** (`nodes[-1]`).

Fix:

```
# Before:
artven = points[nodes[1], 4]   # second node
# After:
artven = points[nodes[-1], 4]  # last node (matches original)
```

Impact: Different material assignments for passes where first and last nodes have different artven values (e.g., at vessel type boundaries).

3.6 Fix 6: Print Instructions Block (HIGH) — `pipeline.py`

Problem: The refactored pipeline ended with only `"X-CAVATE completed in {elapsed}s."` — no operator instructions. The original (lines 5488–5565) printed detailed guidance for nozzle positioning, G92 zeroing, container centering, and multimaterial calibration.

Fix: Added `_print_instructions()` function (∼80 lines) that:

1. Computes network bounding box (min/max x, y, z)

2. Computes travel dimensions (left, right, forward, backward)
3. Computes centering positions: `x_start = (container_x + left - right) / 2`
4. Prints SM instructions (nozzle positioning, G92, G1 to start, Z instructions)
5. Prints MM calibration instructions (calibration tip procedure, offset recording)
6. Prints MM printing instructions (dual-nozzle positioning, axis zeroing)
7. Prints padding report

Validation: Output matches original character-for-character:

```
X_start: 181.45
Y_start: 152.9
Z_start: -271.82
Padding: left=-215.29, right=-215.29, back=-218.18, front=-218.18
```

3.7 Fix 7: Original Network Plot (LOW) — `pipeline.py`

Problem: `create_original_network_plot()` existed in `viz/plotting.py` but was never called from the pipeline. Dead code.

Fix: Added call in `_write_plots()`, passing pre-interpolation `points` and `coord_num_dict`:

```
fig_orig = create_original_network_plot(
    points_original, coord_num_dict_original,
    title="Original Network",
)
fig_orig.write_html(str(config.plots_dir / "network_original.html"))
```

3.8 Fix 8: Flow Rate Default (MEDIUM) — `config.py`, `cli.py`, `gui/app.py`

Problem: The original hardcoded `flow = 0.1609429886081009` at line 3705, overriding the argparse default of `0.1272265034574846`. The refactored code used the argparse default.

Fix: Updated all three locations:

```
# config.py, cli.py, gui/app.py:
flow: float = 0.1609429886081009  # experimentally determined
```

Impact: Speed values now differ by 0% (previously 26.5% difference when no `-flow` arg provided).

3.9 Fix 9: MM Positive Ink Overshoot Adjustment (MEDIUM) — `base.py`

Problem: In multimaterial positive ink displacement mode, when a pass follows a gap closure extension, the G92 re-zeroing coordinates must be adjusted by the extension delta vector. The refactored code computed `prev_x`/`prev_y` from raw coordinates without this adjustment.

Fix: In `_write_network()`, after computing `prev_x`/`prev_y`, check if the previous pass had a gap extension and add the deltas:

```
if gap_extensions and (i - 1) in gap_extensions:
    ext = gap_extensions[i - 1]
    prev_x = round(points[prev_point, 0] + ext.delta_x, nd)
    prev_y = round(points[prev_point, 1] + ext.delta_y, nd)
```

3.10 Fix 10: Aerotech SM Mode (LOW) — `aerotech.py`

Problem: The refactored Aerotech writer always used dual-axis initialization (`G92 X Y A B`) and braked both printheads, even in single-material mode. The original SM Aerotech mode used a single axis and single printhead.

Fix: Added SM-specific code paths in all four methods:

Method	SM Behavior
`_write_first_pass_start`	`G92 X Y A` (single axis), `Enable Aa`, `BRAKE Aa 0`
`_write_pass_start`	`BRAKE Aa 1`, jog, `Enable Aa`, `BRAKE Aa 0`, `DWELL`
`_write_move`	`G1 X Y A(z) F(speed)` (single axis)
`_write_pass_end`	`Enable Aa`, `DWELL`, `BRAKE Aa 1`, lift single axis

Table 4: Aerotech SM-specific code paths added.

3.11 Fix 11: Downsampled and Overlap Plots (LOW) — `pipeline.py`

Problem: The original generated `network_downsampled_SM.html` and `network_SM_overlap.html` when those features were active. The refactored code did not.

Fix: Added conditional plot generation in `_write_plots()`:

```
if was_downsampled:
    fig_ds = create_network_plot(print_passes_sm, points,
                                 title="Downsampled SM")
    fig_ds.write_html(str(config.plots_dir / "network_downsampled_SM.html"))

if has_overlap:
    fig_overlap = create_network_plot(print_passes_sm, points,
                                      title="SM with Overlap")
    fig_overlap.write_html(str(config.plots_dir / "network_SM_overlap.html"))
```

4 Intentional Differences Retained

These differences are **improvements** over the original and were intentionally kept.

4.1 4A. Iterative DFS (Issue 2G)

The original uses recursive DFS, which hits Python's recursion limit on networks with >1,000 interpolated points (31,271 in our test). The refactored code uses an iterative DFS with an explicit stack.

Consequence: Pass compositions may differ (nodes are grouped into different passes), but the same set of nodes is covered. Our test confirms: all 31,271 unique coordinates appear in both outputs.

Why kept: The recursive version literally crashes on the test network without `sys.setrecursionlimit(50000)`.

4.2 4B. Nearest-Neighbor Pass Reordering

The refactored code adds a greedy nearest-neighbor pass reordering step that minimizes total nozzle travel between passes. The original does not reorder.

Consequence: Passes appear in a different sequence, reducing total jog distance.

Why kept: Strictly beneficial — reduces print time without changing the printed geometry.

4.3 4C. KD-tree Inlet/Outlet Matching (Issue 2H)

The original uses exact float comparison (`==`) for matching inlet/outlet coordinates to network nodes. The refactored code uses `scipy.spatial.cKDTree` nearest-neighbor matching with a distance warning threshold.

Why kept: More robust against floating-point precision issues. Exact matching can fail when coordinates have been rounded or scaled differently.

5 Files Modified

File	Lines Δ	Fixes
`pipeline.py`	+156	#1 (artven swap), #6 (print instructions), #7 (original plot), #11 (extra plots)
`postprocessing.py`	+88 −12	#4 (overlap: retrace + consecutive + router)
`gcode/aerotech.py`	+53 −5	#10 (SM mode: single axis/printhead)
`gcode/pressure.py`	+22 −13	#2 (axis naming), #3 (G91/G90 wrapping)
`gui/app.py`	+17 −2	#4 (overlap selector), #8 (flow default)
`config.py`	+9 −2	#4 (OverlapAlgorithm enum), #8 (flow default)
`cli.py`	+8 −2	#4 (`-overlap_algorithm` arg), #8 (flow default)
`gcode/base.py`	+5	#9 (overshoot adjustment)
`multimaterial.py`	+2 −2	#5 (classification node)
Total	**+437 −57**	**11 fixes**

Table 5: All files modified during parity fixes.

6 Validation Results

6.1 Test Configuration

```
python -m xcavate \
  --network_file tree_Test2.txt \
  --inletoutlet_file tree_Test2_inlet_outlet.txt \
  --multimaterial 0 --tolerance_flag 0 \
```

```
  --nozzle_diameter 0.5 --container_height 10 \
  --num_decimals 3 --speed_calc 1 --plots 0 \
  --downsample 0 --custom 0 --printer_type 0 \
  --amount_up 10
```

6.2 Coordinate Coverage

Both versions produce **identical coordinate sets** — every (X, Y, Z) triple in the old output appears in the new output, and vice versa:

```
Old:  31,271 unique coordinates
New:  31,271 unique coordinates
In old but not new: 0
In new but not old: 0
Common: 31,271 (100%)
```

6.3 G-code Structure

Element	Old	New	Notes
Total lines	31,822	31,822	Exact match
Passes	44	45	+1 from explicit Pass 0 comment
Axis name	`A`	`A`	Fixed (was `Z`)
Pass transition	G90/G91	G90/G91	Fixed
Pass end jog	`F10`	`F10`	Fixed (was `F{jog_speed}`)
File size	1.557 MB	1.557 MB	283 bytes diff (whitespace)

Table 6: G-code structural comparison.

6.4 Speed Values

```
Both: 436 unique speed values
Range: 0.25 - 15.676 mm/s
Maximum per-value difference: <0.001 (floating-point precision)
```

6.5 Output Files

6.6 Performance

7 Known Remaining Differences

These are documented and understood:

1. **Pass ordering** — Different due to iterative DFS + nearest-neighbor reordering. Same nodes covered.
2. **Coordinate file line counts** — Differ by 1–45 lines due to different pass groupings.
3. **Changelog verbosity** — Refactored is less verbose (392 vs 1,142 lines).

File	Old	New	Notes
`gcode_SM_pressure.txt`	Yes	Yes	Same structure
`x_coordinates_SM.txt`	31k	31k	+1 line (formatting)
`y_coordinates_SM.txt`	31k	31k	Different pass groupings
`z_coordinates_SM.txt`	31k	31k	+1 line (formatting)
`all_coordinates_SM.txt`	Yes	Yes	
`printspeed_list_SM.txt`	Yes	Yes	
`radii_list_SM.txt`	Yes	Yes	
`graph.txt`	Yes	Yes	
`special_nodes.txt`	Yes	Yes	
`changelog.txt`	1142	392	Less verbose logging
`preprocessed.txt`	Yes	No	By design (in-memory only)
`inlets.txt`	Yes	No	By design (in-memory only)
`outlets.txt`	Yes	No	By design (in-memory only)

Table 7: Output file comparison.

Metric	Old	New
Execution time	∼5 min (with recursion limit hack)	∼2 sec
Recursion safety	Crashes at default limit	No recursion used
Memory pattern	$O(N)$ repeated `np.insert`	$O(N)$ single `np.concatenate`
Collision detection	$O(N)$ full-array scan	$O(k \log N)$ KD-tree
Branchpoint detection	$O(N \times \mathit{endpoints})$ nested loops	$O(N)$ KD-tree query

Table 8: Performance comparison.

4. **Diagnostic files** — `preprocessed.txt`, `inlets.txt`, `outlets.txt` not written (by design; data kept in-memory).
5. **Floating-point precision** — Speed values differ in the last 1–2 digits (e.g., `7.465315064778841` vs `7.46531506477704`). Within machine epsilon; no practical effect on printer behavior.
6. **`arbitrary_val` sentinel** — `999999999` (refactored) vs dynamic value (original). Both are sentinels that never appear in real coordinates.

8 Improvements in the Refactored Package

Improvement	**Detail**
No recursion limit	Iterative DFS with explicit stack — handles any network size
$O(k \log N)$ collision detection	KD-tree `query_ball_point` vs $O(N)$ full-array scan
$O(N)$ interpolation	Single `np.concatenate` vs repeated `np.insert`
KD-tree branchpoint detection	`cKDTree.query()` vs $O(N \times endpoints)$ nested loops
Min-heap for lowest unvisited	$O(\log N)$ vs $O(N)$ linear scan
Fuzzy inlet/outlet matching	cKDTree nearest-neighbor vs exact float `==`
Vectorized preprocessing	`np.round()` on arrays vs triple-nested Python loops
Configuration dataclass	`XcavateConfig` replaces 30+ global variables
G-code writer ABC	Eliminates duplication across 6 G-code variants
Strategy pattern	Pluggable pathfinding algorithms (DFS + sweep-line)
Nearest-neighbor reordering	New feature minimizing nozzle travel
Streamlit GUI	Web interface for non-programmers
Structured logging	Python `logging` module vs raw `print()`
Custom codes loaded once	vs re-opening files on every pass start/end
Test suite	501 lines of pytest tests covering core modules

Table 9: Improvements in the refactored package.